

ByteMage Employee Knowledge Base

A fictional sample document for experimenting with document parsing, chunking, overlap, metadata, lexical retrieval, vector retrieval, hybrid search, and reranking.

1. Company Overview

ByteMage is a fictional software company that builds cloud applications, data platforms, and AI-powered business tools. The company operates engineering, product, sales, finance, and customer-success departments. Its engineering organization focuses on reliable APIs, scalable web applications, search systems, and machine-learning infrastructure.

2. Employee Profile

John Doe is a Senior Software Engineer in the AI Platform department. He joined ByteMage in March 2022 and works primarily on retrieval-augmented generation systems, search infrastructure, and internal developer tools. John's employee ID is BM-1047 and his manager is Sarah Ahmed.

3. Education and Background

John completed a Bachelor of Science in Computer Science from the University of Lahore in 2018. Before joining ByteMage, he worked as a backend engineer at DataWorks, where he developed Python services and PostgreSQL-based applications. He later moved into AI engineering and began working with embeddings, vector databases, Elasticsearch, and large language models.

4. AI Search Platform

ByteMage's AI Search Platform combines lexical retrieval and semantic retrieval. Lexical retrieval uses Elasticsearch and BM25 to identify documents containing important query terms. Semantic retrieval uses embedding vectors to find passages with similar meaning even when the query and document use different vocabulary. The platform combines these approaches through hybrid retrieval and then applies a reranking stage to improve the ordering of candidate passages.

5. Chunking Strategy

Documents are divided into smaller chunks before indexing. The default strategy uses approximately 400 tokens per chunk with an overlap of 60 tokens. The overlap helps preserve context when an important sentence spans a chunk boundary. For structured documents, headings and section boundaries are retained as metadata so that retrieved chunks can be traced back to their original location.

6. Search Metadata

Every indexed chunk stores a stable chunk ID, the original text, source filename, title, section, document type, creation date, department, and parent document ID. Identifiers such as employee IDs and product codes are stored as keyword fields when exact filtering is required. Long natural-language content is stored as text fields so that Elasticsearch can analyze and tokenize it for full-text search.

7. Retrieval Example

Consider the question: Where did John study? A semantic retriever may return passages discussing John's education because they are conceptually related. A lexical retriever may prioritize the exact terms John, study, Computer Science, or University. The hybrid system merges candidates from both retrievers, removes duplicate chunks, and uses rank-based fusion to produce a broader candidate set.

8. Reranking

The reranker receives the user's query together with each candidate passage. Its purpose is to judge how useful each passage is for answering the exact question. A passage can be semantically similar without directly answering the question, so reranking improves final precision. The system normally retrieves more candidates than it ultimately sends to the language model; for example, it may retrieve 20 candidates and keep the best 5 after reranking.

9. Exact Identifiers

Exact-match retrieval is especially important for identifiers such as BM-1047, ticket IDs, product codes, invoice numbers, and document IDs. These values should not rely only on semantic similarity. Keyword fields and term queries can preserve the exact identifier and avoid returning a merely similar value.

10. Evaluation

Retrieval quality should be evaluated separately from answer generation. Candidate recall asks whether the correct chunk entered the candidate set. Final precision asks whether the highest-ranked chunks are actually useful. If the correct chunk was never retrieved, reranking cannot recover it. Useful experiments include changing chunk size, overlap, retriever weights, candidate count, and reranking depth, then comparing retrieval metrics and latency.